

Отчет о разработке модели детекции переломов на рентгеновских снимках

Галицына Алиса

15.07.2026

Введение

Задача: детекция переломов на рентгеновских снимках.

1 Метрики

1.1 Precision (Точность)

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1)$$

где TP — количество истинно положительных обнаружений, FP — количество ложноположительных обнаружений.

Precision показывает, какая доля предсказанных моделью переломов действительно является переломами. Высокая точность означает, что модель редко ошибается ложными срабатываниями.

1.2 Recall (Полнота)

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

где FN — количество ложноотрицательных обнаружений (пропущенных переломов).

Recall показывает, какую долю от всех реальных переломов модель смогла обнаружить. Высокая полнота означает, что модель пропускает мало переломов.

1.3 F1-Score

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

F1-Score является гармоническим средним между Precision и Recall и позволяет оценить баланс между этими двумя метриками.

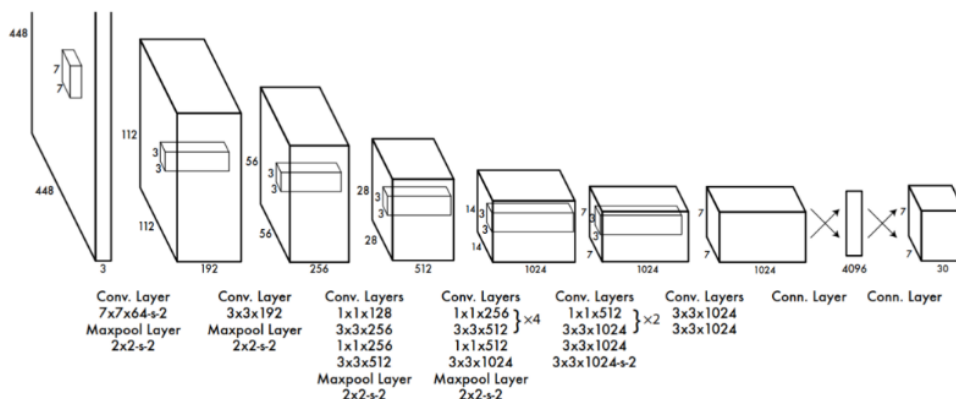
1.4 mAP@50

$$\text{mAP@50} = \frac{1}{N} \sum_{i=1}^N AP_i \quad (4)$$

где AP_i — площадь под кривой Precision-Recall для каждого класса, а IoU порог установлен на 0.5.

2 Решение

Возможные решения: мною было решено реализовать архитектуру YOLOv1 для задачи детекции переломов на рентгеновских снимках.



3 Первый этап. Базовая архитектура YOLOv1

Обучение производилось на 50 эпохах. Я использовала оптимизатор Adam.

Я использовала операцию Batch Norm после каждого сверточного слоя.

Во время обучения нейронной сети значение функции потерь колебалось около 2.7 и ниже не падало.

3.1 Анализ возможной причины

Возможная причина: дисбаланс классов. В каждом изображении большинство ячеек сетки не содержат объектов, и сеть "училась" предсказывать отсутствие переломов, игнорируя редкие объекты, как переломы.

3.2 Модификации для улучшения сходимости

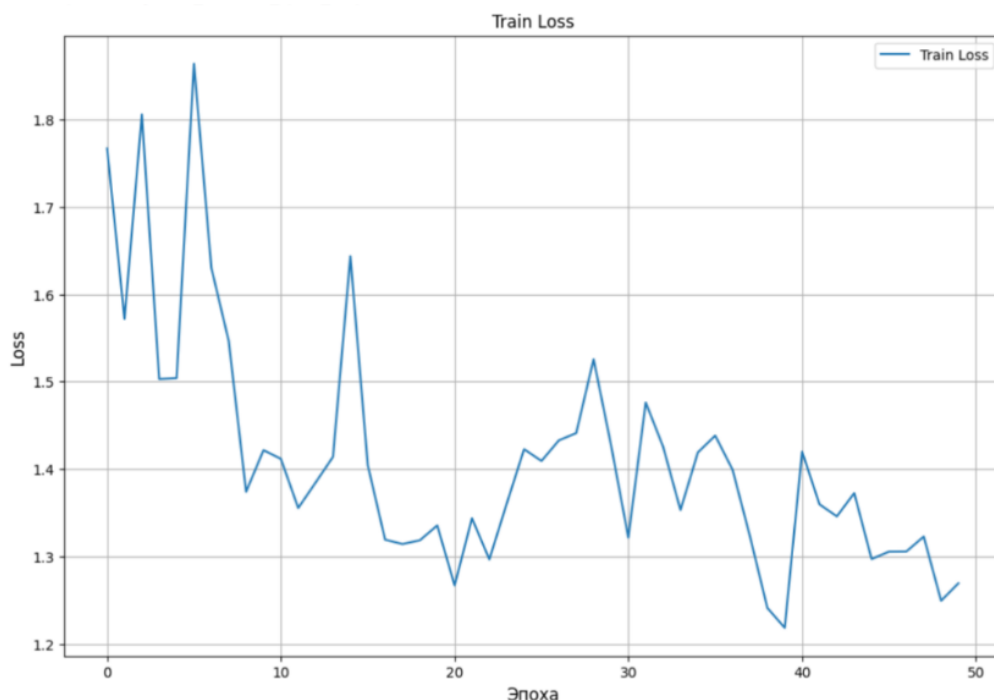
Я добавила динамическое изменение learning rate (чтобы не застревать в локальных минимумах), аугментацию данных для изображений с переломами. Также мною был добавлен Gradient Clipping и нормализация высоты и ширины (это было сделано для предотвращения взрывов градиента, так как во время обучения возникали значения лосса в районе 70).

Код:

```
self.transform_fracture = A.Compose([
    A.Resize(height=448, width=448),
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.5),
    A.RandomRotate90(p=0.5),
    A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ToTensorV2()
], bbox_params=A.BboxParams(format='albu', label_fields=['class_labels']))
```

3.3 Результат

В результате внесенных изменений значение лосс-функции упало до 1.25, но обучение было довольно нестабильным (резкие скачки значений).



4 Второй этап. Замена полносвязных слоев на сверточную голову

Далее вместо полносвязных слоев я решила использовать сверточную голову (по аналогии с YOLOv2). Цитата: "We modify this network for detection by removing the last convolutional layer and instead adding on three 3×3 convolutional layers with 1024 filters each followed by a final 1×1 convolutional layer with the number of outputs we need for detection." Источник: <https://github.com/GitYCC/machine-learning-papers-summary/blob/master/cv/yolo-v2.md> — для лучшего запоминания пространственной информации.

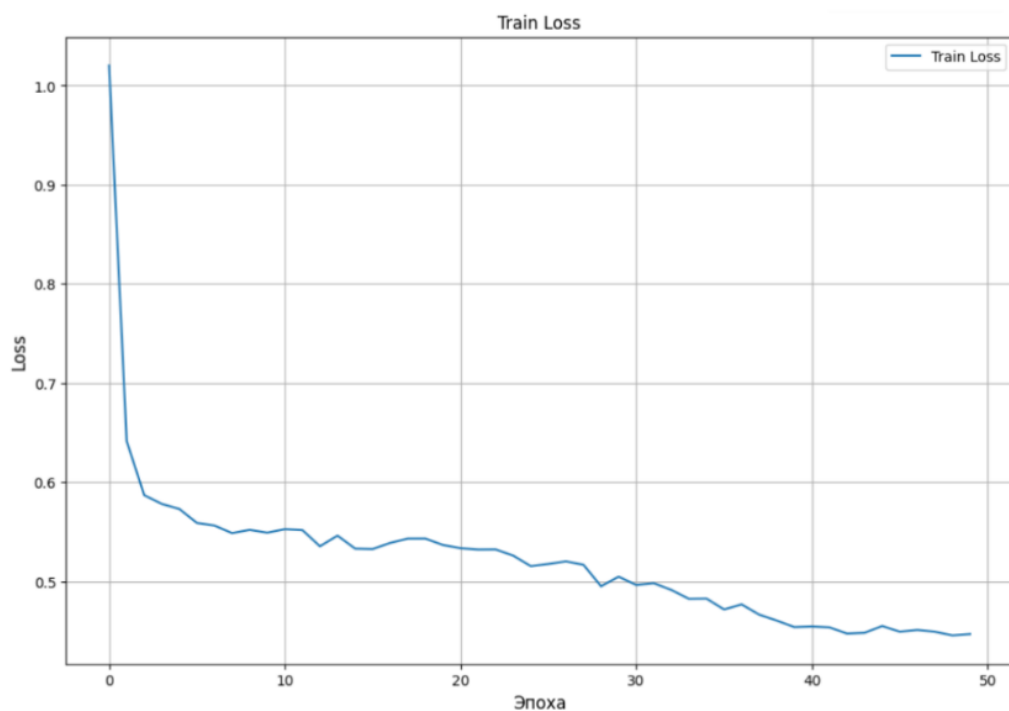
Также я добавила адаптивное изменение весов: я перестала фиксировать `l_no_obj` и сделала его динамическим в зависимости от текущего соотношения объектов и пустых ячеек.

В коде: `dynamic_no_obj_weight = self.l_no_obj * (1.0 - obj_ratio * 0.5)` — когда объектов мало (`obj_ratio` маленький), вес становится больше, чтобы сеть не игнорировала пустые ячейки; когда объектов много — вес уменьшается, чтобы пустые ячейки не доминировали.

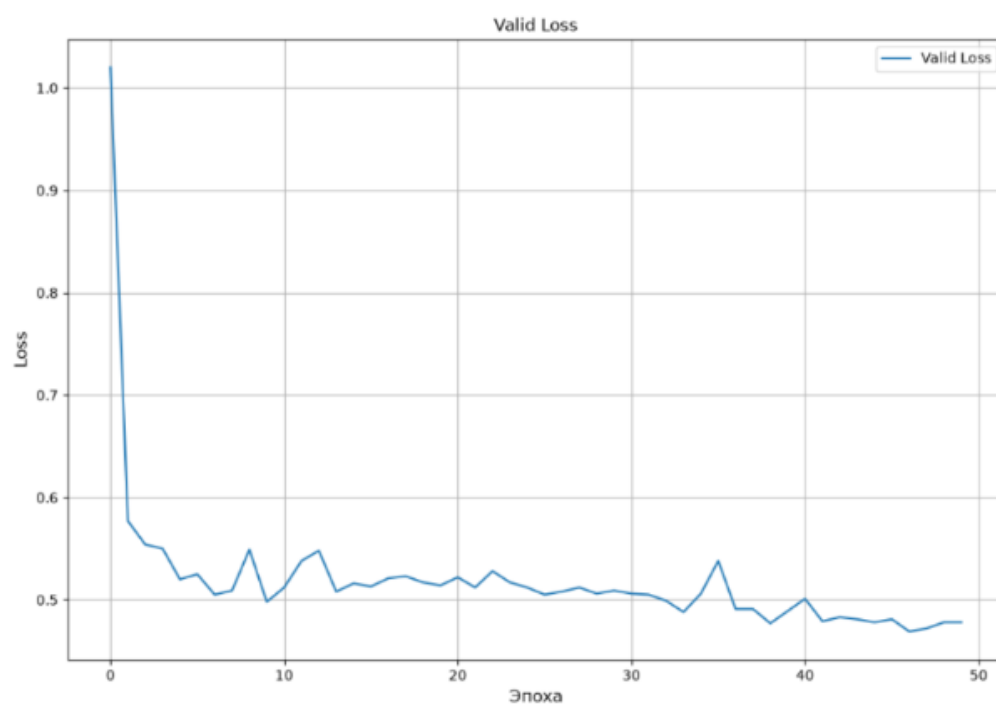
Это было необходимо, потому что в каждом изображении большинство ячеек сетки не содержат объектов, и поэтому сеть будет везде предсказывать отсутствие объектов.

4.1 Результат

В результате данных изменений обучение стало стабильнее и лосс упал до 0.45.



При этом на валидационной выборке лосс тоже падает, а значит модель не переобучается.



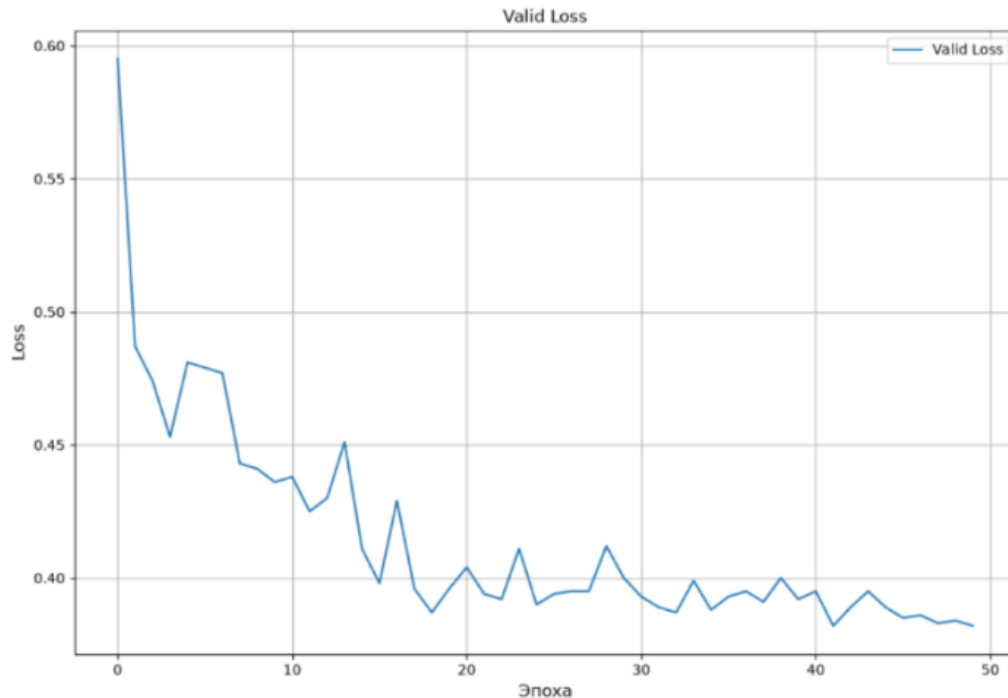
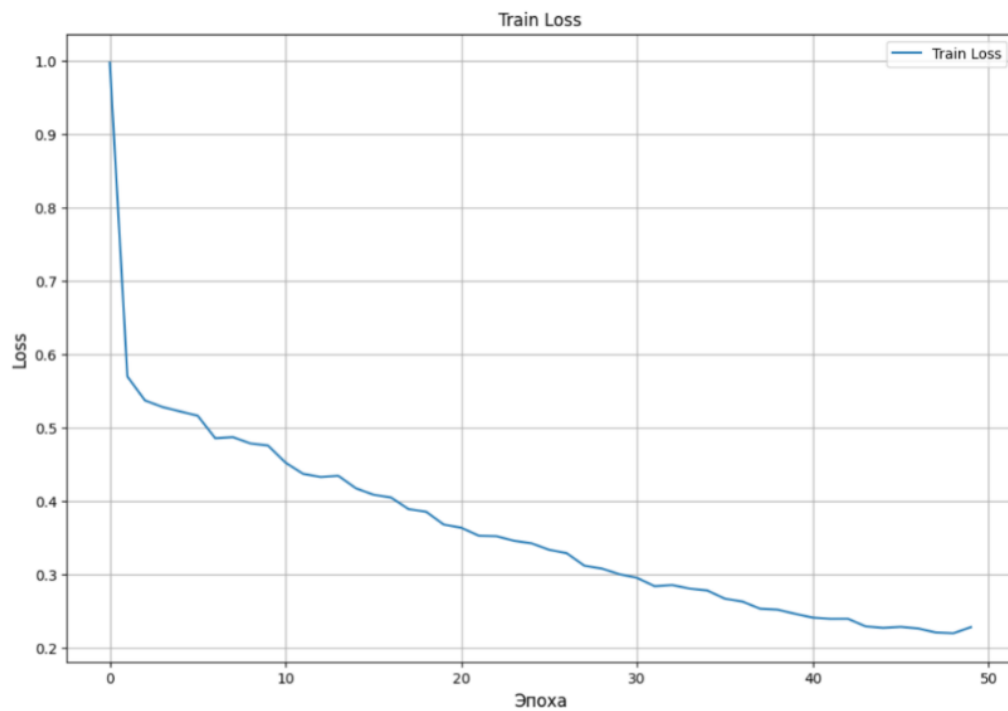
5 Третий этап. Использование предобученной ResNet и FPN

Далее я добавила вместо сверточных слоев предобученную ResNet:

```
resnet = models.resnet34(weights=models.ResNet34_Weights.DEFAULT)
self.backbone = nn.Sequential(*list(resnet.children())[:-2])
```

Значение лосса на трейне упало до 0.2, но начиная с 20-й эпохи лосс на валидационной выборке начал расти.

Поэтому я добавила Dropout(0.3) и упрощенные FPN слои. Лосс на трейне стал 0.23, при этом на валидации падал.



5.1 Метрики для данного запуска

Метрики для данного запуска получились следующие:

- Precision: 0.0306 (при пороге 0.05)
- Recall: 0.1535

- F1-Score: 0.0510
- mAP50: 0.1195

Precision показывает, сколько из предсказанных переломов действительно переломы. Очень низкое значение (0.03) означает, что модель даёт огромное количество ложных срабатываний.

Recall (0.15) говорит, что модель находит лишь 15% реальных переломов.

F1 (0.05) — гармоническое среднее, отражает общее низкое качество.

mAP50 (0.12) — тоже очень низкая.

5.2 Возможные причины низких метрик

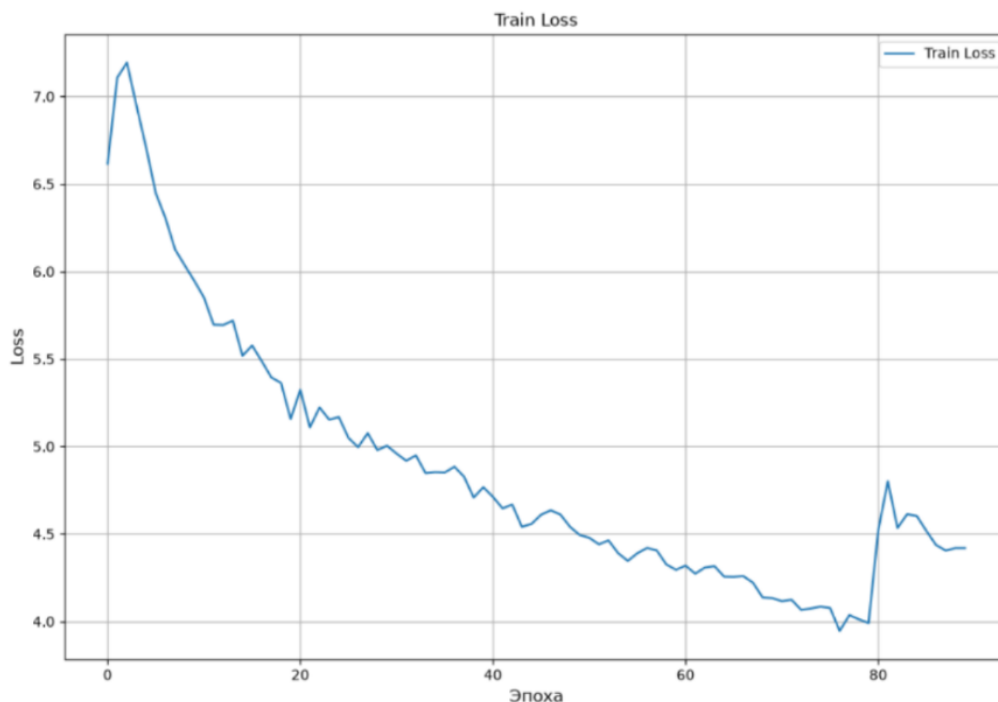
Возможные причины низких метрик: модель предпочитает не предсказывать перелом, чем ошибиться. При этом разрешение 448×448 YOLOv1 может быть недостаточно детальным для мелких переломов. Сетка может быть слишком крупной, так как переломы обычно небольшие.

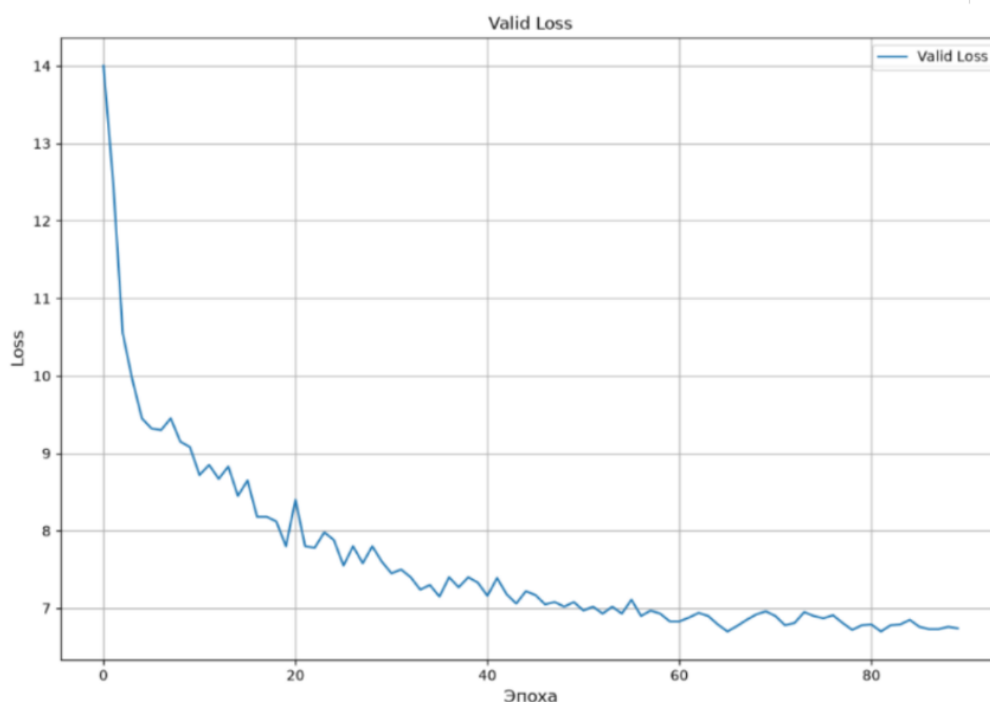
6 Вторая архитектура. YOLOv8

Вторая архитектура: я использовала архитектуру YOLOv8.

Дополнительно мною была написана функция, которая приводит все классы переломов к единому значению. Также я очистила кэш, чтобы после исправления разметки модель не использовала старую. Также мною был создан файл data.yaml для конфигурации датасета при обучении модели YOLO. В нем указываются пути к изображениям и разметке, количество классов и их названия.

Обучение производилось на 90 эпохах.





6.1 Особенности обучения

На графике с train loss виден скачок на 81-й эпохе. Это происходит из-за mosaic-аугментации, которая объединяет 4 изображения в одно, делая изображение более сложным. Когда mosaic отключается, модель переключается на реальные изображения, и поэтому происходит временный рост потерь, так как модель должна адаптироваться к другому распределению данных. Но на валидации лосс продолжает падать! Это было добавлено для того, чтобы модель лучше обобщала данные.

6.2 Полученные метрики YOLOv8

Полученные метрики:

- Precision: 0.5643
- Recall: 0.2990
- F1-Score: 0.3909
- mAP50: 0.3471

Precision уже значительно выше — модель реже ошибается ложными срабатываниями. Recall тоже выше, чем у первой модели, но всё ещё невысок — значит, около 70% переломов остаются незамеченными. F1 и mAP50 тоже лучше, чем в первой версии.

6.3 Что можно улучшить

Что можно улучшить:

- увеличить число эпох после отключения Mosaic, так как мы видим, что потери продолжают падать;
- попробовать разные размеры входных изображений;
- добавить больше данных с переломами.